Token-Aware Agentic Architecture for Cospas-Sarsat Alert Intelligence: Predictable Cost, Protected Safety

Smruti Rout

Software Architect & AI Researcher
Bengaluru, Karnataka, India
Email: smruti.srr@gmail.com
White Paper — October 2026

Reference Use Case: An Alert Intelligence Agent at 8,000 Assessments per Day

***Abstract*—AI agents can support first-line triage of Cospas-Sarsat distress alerts, but an agent re-submits its goal and history at every step, and every tool call adds another full re-read, so token cost grows with every alert handled. This paper proposes a token-aware agentic control plane that combines tiered model routing, prompt caching and context compaction with automatic escalation, outcome-based cost telemetry and a read-only tool gateway. The agent runs beside the alert path, is advisory only, and fails open to a human controller. Using published list prices and a transparent four-step pipeline at 8,000 assessments per day, the modelled cost of a tool-free workflow falls from $0.1182 to $0.0376 per assessment, a 68.2% reduction worth about $235,000 a year. Letting the correlation step call tools nearly doubles the cost of the cheap design, to $0.0721 per assessment, yet that is still 72.2% below a naive premium agent with the same tools (about $547,000 a year), provided the gateway trims results, caching stays intact and a ceiling bounds the number of tool rounds. The savings survive pessimistic cache and failure assumptions, and compaction overtakes caching on long investigations. Prices are 2024-era and scenario inputs are assumptions to be replaced with pilot data. The paper also sets out safety invariants, governance metrics, a phased roadmap and a reproducible calculation script.**

***Index Terms*—Cospas-Sarsat, search and rescue, LLM agents, tool use, token economics, prompt caching, model routing, FinOps**

I. Introduction and Summary of Findings

An AI agent that triages Cospas-Sarsat alerts can cost 68% less than a premium-model-only design without touching the safety path: in our modelled deployment, spend falls from $0.1182 to $0.0376 per assessment, about $235,000 a year at 8,000 assessments a day. Letting that agent call tools raises the bill, but the same disciplines keep it 72% below a naive tool-using agent (Section VI).

Independent analysis. This paper is not an official Cospas-Sarsat publication, and the architecture it describes is a proposal, not a programme specification. Every cost figure is reproducible with the accompanying Python script (Appendix A). Dollar figures use 2024-era list prices for the models named; re-run the script with current prices before using them for budgeting.

The case for triage support is strong. The Cospas-Sarsat System turns 406 MHz beacon activations into alert and location data for rescue authorities, and it has assisted in rescuing about 67,000 people since 1982, to the end of 2024 [6]. Yet the programme's own statistics show that most alerts reaching SAR services are not genuine emergencies: in 2019 the average SAR false alert rate, which counts false and undetermined alerts together, was 96.65%, roughly one real alert in 30 [9]. First-line verification is therefore a large share of a rescue coordinator's work, and it must be done while treating every alert as real until proven otherwise.

AI agents are well suited to that evidence-gathering work, but they are expensive in a way chatbots are not. An agent re-submits its goal, scratchpad and prior reasoning at every step. A 2026 Stanford Digital Economy Lab study of agentic coding tasks found that agents consumed about 1,000 times more tokens than code reasoning or chat, and that repeated runs of the same task differed by up to 30 times in total tokens [1]. Those are coding benchmarks, not alert triage, but the mechanism is the same: the whole context is re-read at every step. Left ungoverned, that cost scales with every alert.

This paper applies a *Token-Aware Control Plane* to the problem: intelligent model routing, prompt caching and context compaction, backed by automatic escalation, outcome-based FinOps telemetry and a read-only tool gateway for the steps where the model chooses its own evidence. Using published API prices and a transparent four-step pipeline, I find:

- **68.2% lower cost** than a premium-by-default design (the tool-free workflow of Sections III to V): $345,144 falls to $109,679 per year, a saving of **$235,465** for one workflow.
- **Routing alone saves 34.6%.** Prompt caching removes a further **51.4%** of what remains, because the SAR procedures prompt is large (6,000 tokens) and identical on every run.
- **The failure accounting is conservative.** Failed cheap-model attempts are billed in full, and the premium base-

line is given no failure penalty. The resulting escalation tax is 20.4%. Two other assumptions run the other way (a 100% cache hit rate and no cache-write fee), which is why Section V-D stress-tests them.

- **The saving is robust.** It stays at 56.6% even if the cheap model fails 60% of the time, and at 51.4% if the cache serves only half of the static prompt. Routing alone stops paying only when the cheap model fails about 80% of the time.
- **For long investigations, compaction overtakes caching.** On a 40-step case-monitoring agent, caching saves 28%, compaction 35%, and the two together 63%.
- **Tools nearly double the cheap design's bill, and the design still pays.** Letting Step 2 call tools through a read-only gateway raises the cost from $0.0376 to $0.0721 per assessment (+91.9%). That is 72.2% below a naive premium agent with the same tools ($210,463 against $757,740 a year) and 39.0% below the tool-free premium workflow. A tool-call ceiling bounds the worst case at 2.3 times the mean.

Cost control must never compete with safety. The architecture keeps the agent **beside the alert path, never in it**; humans make every decision; no step that judges distress likelihood runs on the cheapest tier; every tool is read-only; and when any budget limit is hit the system **fails open to a human controller** with the raw alert intact.

The lesson for leaders is that token volume is a cost metric, not a value metric. The measure that matters is cost per assessed alert, set against the controller time and certainty it buys.

II. Context: Why Alert Triage Is a Token Problem

A. How the Alerting Chain Works

Cospas-Sarsat is an intergovernmental programme of 45 Participants (May 2025) [5]. Beacons such as EPIRBs, ELTs and PLBs transmit at 406 MHz; satellites in low-Earth, geostationary and medium-Earth orbit relay the signals to ground stations; Mission Control Centres (MCCs) distribute the resulting alert data to Rescue Coordination Centres (RCCs) and other SAR points of contact. Cospas-Sarsat provides the data. It does not itself conduct search-and-rescue operations. The Secretariat also maintains the International Beacon Registration Database (IBRD), which lets responders identify who owns a beacon and whom to call. The medium-Earth-orbit (MEOSAR) system reached its initial operational capability in April 2023 [8].

The human impact is concrete. In 2024, Cospas-Sarsat alert data assisted in 1,171 distress incidents and 3,211 persons were rescued [5]. These are the rescues; behind them sits a far larger flow of alerts that turn out not to be emergencies.

B. The Triage Burden

Cospas-Sarsat measures a SAR false alert rate: false alerts plus undetermined alerts (no person in distress found, or no beacon found) as a share of alerts transmitted to SAR services. The rate was stable from 2015 to 2019, between 96.30% and 96.85%, averaging 96.65% in 2019 [9], and was 95.55% in 2020 [10]. Earlier status papers note that ELT beacons contribute disproportionately, and that correct beacon registration greatly improves the chance of resolving an alert quickly. About 3.17 million 406 MHz beacons were deployed in 2023, roughly three-quarters of them GNSS-capable [7], so the volume of alert data to reconcile is large and steady.

One principle governs all of this work: **an alert is treated as real until it is confirmed false.** That principle shapes our architecture. An AI agent may enrich, rank and recommend. It may never close, suppress or delay.

C. Why Agents Break the Chatbot Cost Model

A chatbot is one query, one call, one answer. An agent is a loop, and three mechanics change the economics:

1) **State carrying.** Each step resubmits the whole history, including the goal, prior reasoning and tool results, so input context grows with every step and cumulative billed tokens grow faster than the step count.
2) **Input-dominated cost.** In the premium-by-default baseline of Section V-B, input tokens account for 77.2% of cost, because the agent keeps re-reading procedures, schemas and registry records to make small decisions. The Stanford study reports the same pattern in coding agents [1].
3) **Stochastic volatility.** Depending on whether the agent finds an efficient path or gets stuck in a reasoning loop, the same job can vary by up to 30 times in total tokens [1]. Budgeting on averages alone is unsafe.

D. Three Traps

- **The leaderboard misconception.** Ranking people or teams by tokens consumed rewards burning tokens, not resolving cases.
- **Premium by default.** Wiring every step to the strongest model is simple, and at scale most steps (parsing, formatting) never needed it.
- **Savings that touch safety.** The most dangerous optimisation is the one that quietly lowers the quality of a life-safety judgement. Section IV-G sets out invariants that cost control may not cross.

III. The Use Case: An Alert Intelligence Agent

A. Scenario

A regional hub (an MCC and its supported RCCs) deploys an agent that performs first-line assessment each time a new alert arrives or a material update changes an existing one. It returns a structured case brief that a human controller reviews: who the beacon is registered to, whether it has a history of false activations, how it correlates with other detections and context, how likely the alert is to be genuine, and which verification actions to take first. The alert itself is forwarded to the RCC at once, exactly as it is today; the brief follows alongside it. The pipeline has four steps (Table I).

Design note on Step 1. Where alert messages arrive in a fixed, specified format, parse them with deterministic code and

TABLE I
THE FOUR-STEP ASSESSMENT PIPELINE

Step	Task	Detail	Complexity	Tier
1	Parse	Decode and normalise the alert message; classify beacon type and activation context	Low (extraction, classification)	Tier 1 (lite)
2	Correlate and assess	Assess the prefetched registry record and history; call read-only tools for further evidence (other detections, vessel or aircraft track, weather, drift) as the findings require; estimate likelihood of genuine distress; recommend checks	High (multi-source reasoning, ambiguity, tool use)	Tier 2 (core)
3	Structure	Build the schema-valid case payload for the RCC case-management system	Low to medium, schema-sensitive	Tier 1 (lite)
4	Verify	Consistency and safety validation before a controller sees the brief; confirm nothing was suppressed	High (safety-critical)	Tier 2 (core)

pass the result to the model as data. Reserve the Tier 1 model for normalisation and classification, so that a misread position or beacon identifier cannot originate in a language model.

Workflow versus agent. Steps 1, 3 and 4 run in a fixed order, so the pipeline is a routed workflow: the router chooses the tier and the controller decides on retries, but the model never chooses what happens next. Step 2 is where autonomy lives. It is a bounded observe-and-act loop in which the Tier 2 model decides which read-only tool to call next, reads the result, and decides whether it has enough evidence to write the assessment, all inside an allowlist, a per-run tool-call ceiling and the safety invariants of Section IV-G. Evidence that every assessment needs (the registry record and the beacon's history) is prefetched by deterministic code and included in the 900-token goal. Tools are for follow-ups whose need depends on what was found. The figures in Sections V and VII price this prefetched workflow, which is the cost floor; Section VI prices Step 2 with tool calls enabled.

B. Why This Use Case Is Representative

It has every feature that makes agent spend hard to control: a heavy static prompt (SAR procedures, output schemas, false-alert playbooks), a mix of easy and hard steps, a strict output schema where cheap models sometimes fail, and high, steady, around-the-clock volume.

C. Modelling Assumptions

The scenario inputs are illustrative assumptions, not measured Cospas-Sarsat figures. They are stated so that any operator can replace them with pilot data.

- Static prompt of 6,000 tokens (procedures, schemas, playbooks) and a dynamic goal of 900 tokens (decoded alert, registry record, history).
- Output per step of 300, 700, 500 and 300 tokens; each step's output is carried into the next step's input.
- Volume of 8,000 assessments per day, 365 days a year (2.92 million runs).
- First-attempt failure probability of 15% on each Tier 1 step and 5% on each Tier 2 step. A failed Tier 1 step is re-run on the premium model with 40% more output.
- **Failed attempts are billed.** The wasted first attempt is charged in full, which is more conservative than a simple simulator that does not charge for failed attempts (Section V-F).
- Premium-by-default baseline: all four steps on the premium model, no caching, and no failure penalty.
- Prompt caching applies to the static prefix only, with a 100% hit rate in the base case (Section V-D varies it).
- Expected costs are computed exactly by enumerating all 16 failure outcomes, then cross-checked with a 200,000-run Monte Carlo simulation.

IV. REFERENCE ARCHITECTURE: THE TOKEN-AWARE CONTROL PLANE

The control plane sits between the alert-intelligence agent and its model providers and data sources, and makes every model call and tool call cheaper, more reliable and more observable without changing the agent's business logic. It runs beside the alert-forwarding path, never on it. Fig. 1 shows the two paths: the safety-critical alert path at the top, which is unchanged and human-decided, and the advisory assessment path below it, which receives only a copy of each alert and fails open. Tool calls from Tiers 2 and 3 pass through the tool gateway (component 6) to read-only data sources; Tier 1 has no tool access.

A. Component 1: Router Evaluation Layer

The router sends each step to the cheapest model that can handle it safely. It uses three inputs: intent classification (extraction versus assessment), predicted input length, and confidence and schema-validation signals that trigger escalation. A fourth input is specific to this domain: **severity indicators** such as an encoded position, a registered vessel or aircraft in a hazardous setting, or corroborating detections. These raise the floor tier and lift budget limits, including the tool-call ceiling (Section IV-G).

Table II lists the tiers. Tier 3 is deliberately left out of the cost figures in Section V because the published pricing gives no fixed price for it. Its use is expected to be rare, and it should be reported as a separate line when measured.

B. Component 2: Prompt Cache Manager

The SAR procedures, output schemas and false-alert playbooks (6,000 tokens in our scenario) are identical on every call.

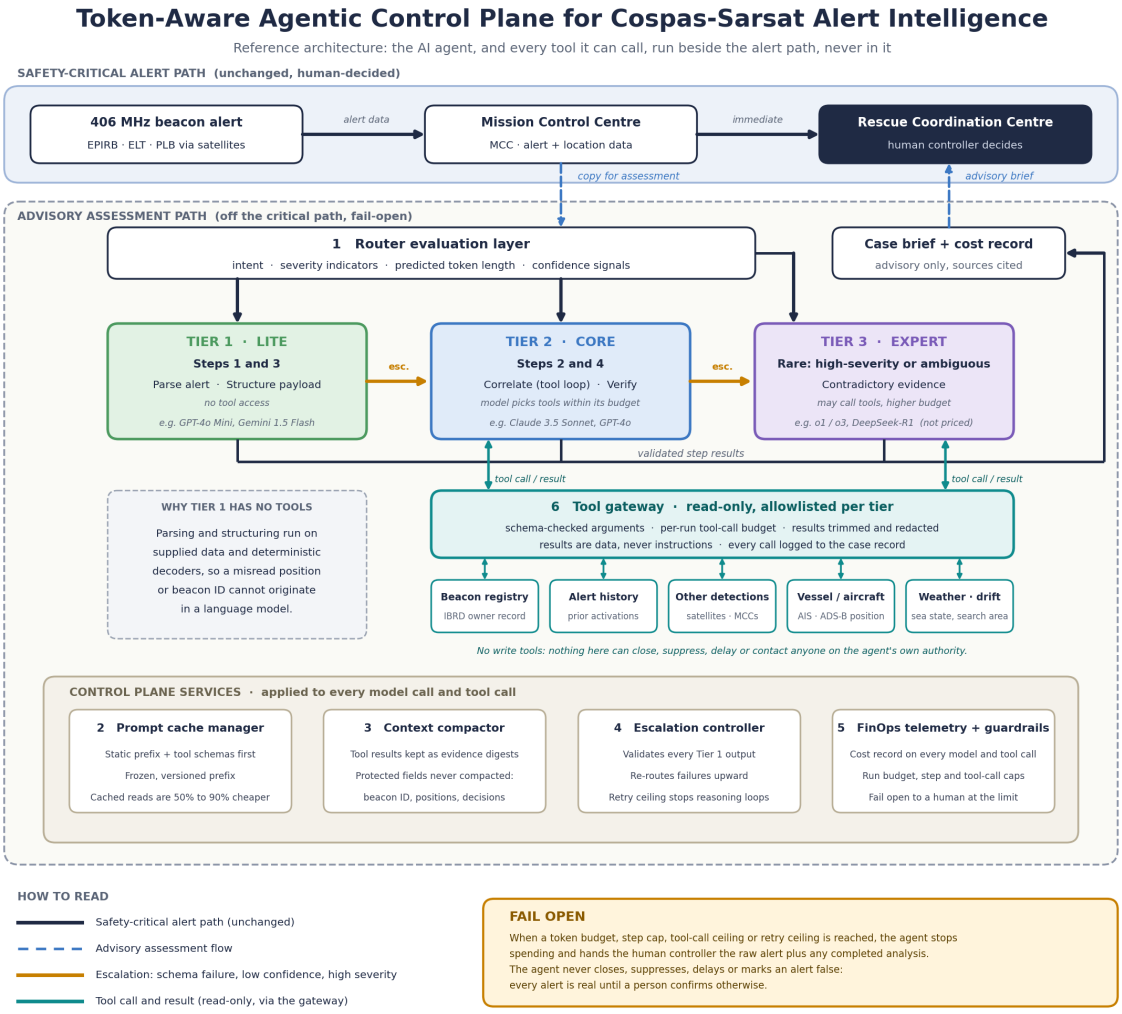


Fig. 1. The token-aware control plane. The agent runs beside the alert path, never in it. Numbers 1 to 6 match the components described in Sections IV-A to IV-F. Tier 1 has no tool access; Tiers 2 and 3 reach read-only data sources only through the gateway.

TABLE II
MODEL TIERS AND INDICATIVE COST PROFILES

Tier	Sample models	Intended work in this use case	Cost profile (per 1M tokens, list price)
Tier 1: Lite	GPT-4o Mini, Gemini 1.5 Flash	Alert parsing and normalisation, case-payload structuring	$0.15 to $0.35 input; $0.60 to $1.50 output
Tier 2: Core	Claude 3.5 Sonnet, GPT-4o	Correlation, likelihood assessment, safety verification	$2.50 to $3.00 input; $10.00 to $15.00 output
Tier 3: Expert	OpenAI o1/o3, DeepSeek-R1	Rare, high-severity or contradictory cases	Variable, with a high compute premium

Prompt caching stores that prefix so the agent pays a fraction of the input price each time it is re-read. The listed discounts are 50% on cached inputs for OpenAI models and 90% for Claude 3.5 Sonnet. The manager enforces one discipline: **static content first, dynamic content last**, with the static prefix versioned and frozen between controlled releases so that the cache stays valid. The tool definitions (about 1,200 tokens in our scenario) are static too, so they join the frozen prefix and are cached with it.

C. Component 3: Context Compactor

Because state carrying makes input grow with every step, the compactor periodically summarises older history before the next call. It operates only on a defined compactable region. **Protected fields** (beacon identifier, position and time of each detection, prior assessments and controller decisions) are never summarised away. Tool results are the fastest-growing part of the context, so once a result has been used the compactor replaces it with an evidence digest that keeps its source, times-

tamp and protected fields. Section VII shows why compaction matters for long-running investigations.

D. Component 4: Escalation and Fallback Controller

Cheap models are cheap because they are less capable, and they will sometimes fail. The controller validates every Tier 1 output against its schema and assertions. On failure it halts the step, logs the event, and re-routes the exact context payload to a premium model. Tier 2 steps that return low confidence, contradictory evidence or a high-severity indicator escalate to Tier 3 in the same way. A retry ceiling prevents reasoning loops. Every escalation is a recorded signal that the step may be mis-scoped.

E. Component 5: FinOps Telemetry and Guardrails

Every model call emits a cost record: model, uncached input, cached input, output, step, run and outcome. Every tool call emits a matching record: tool, result size, latency and outcome. Guardrails use these records to enforce per-run token budgets, step caps, tool-call ceilings and alerts. They are the defence against the 30-times volatility problem, because the worst-case run is bounded and visible rather than discovered on the monthly invoice.

F. Component 6: Tool Gateway

An agent differs from a workflow in one respect: the model chooses actions. In this architecture the only actions are read-only lookups, and the model never reaches a data source directly. Every call passes through the tool gateway, which sits between Tiers 2 and 3 and the data sources (Fig. 1). Tier 1 has no tool access: parsing and structuring run on supplied data and deterministic decoders, so a misread position or beacon identifier cannot originate in a language model. Table III shows an illustrative tool set. Evidence that every assessment needs is prefetched by code from the decoded beacon identifier; the model calls tools only for follow-ups whose need depends on what it has found.

The gateway does six jobs:

1) **Least privilege.** Read-only credentials and an allowlist per tier and per step. No tool can write, close, suppress, send a message or contact anyone.
2) **Checked arguments.** Arguments are validated against a schema, and personal data is redacted before any call leaves the hub's approved region.
3) **Normalised results.** The gateway returns only the fields the step needs, at a bounded size, with a source and a timestamp. In the model of Section VI this trims a raw 1,500-token result to 500 tokens.
4) **Results are data.** Tool output is delimited and passed as data, never as instructions. Text in a result that looks like an instruction is ignored and logged.
5) **Budget and timeouts.** A per-run tool-call ceiling (eight rounds in our model, higher for severe alerts) and per-call timeouts, enforced by the controller and not by the model. Reaching the ceiling fails open (invariant 4).
6) **Evidence ledger.** Every call and result is logged to the case record, and a compact digest goes forward to later steps, so the verifier and the controller can trace each statement in the brief to its source.

Who decides what. The router decides which tier runs a step. The Tier 2 model decides which allowed tool to call next and when it has enough evidence. The controller decides when the budget is spent. A human decides everything that matters for the alert. Only the second of these is autonomy, and it is confined to choosing among read-only lookups.

G. Safety Invariants That Cost Control May Not Cross

These eight rules take precedence over every optimisation in this paper.

1) **Off the critical path.** The agent never gates, delays or replaces alert forwarding. If the agent is slow, wrong or down, controllers still receive every alert exactly as they do today.
2) **Advisory only.** The agent enriches, ranks and recommends. A human controller makes every decision, and no alert is closed, suppressed or marked false by the agent.
3) **Severity floor.** Steps that judge likelihood of distress (assessment and verification) never run on Tier 1. Alerts with severity indicators are routed to at least Tier 2, and ambiguous ones to Tier 3. Higher-severity alerts also receive a higher per-run token budget and tool-call ceiling, so cost limits never starve the cases that matter most.
4) **Fail open.** When a token budget, step cap, tool-call ceiling or retry ceiling is reached, the system stops spending and hands the human controller the raw alert plus whatever analysis is complete. It never aborts silently or drops the case.
5) **Asymmetric error cost.** A genuine distress ranked as low priority is the failure that matters most. Validation tracks this recall separately, and a release is blocked if any genuine alert in the validation set falls below the controller-review threshold (Section VIII explains how to size that set).
6) **Degraded mode is normal operations.** The control room must be able to work with the agent switched off, and the model layer must support failover across providers.
7) **Read-only, least-privilege tools.** The agent has no write, send, close or contact capability. Tools are allowlisted per tier and step, arguments are schema-checked and redacted, and tool output is treated as untrusted data, never as instructions.
8) **Evidence or silence.** Every factual statement in a brief cites a tool result or a supplied record. A tool that fails, times out or returns nothing is reported to the controller as missing evidence; the agent never fills the gap from model memory.

TABLE III
ILLUSTRATIVE READ-ONLY TOOL SET

Tool	Returns	How it is invoked
Beacon registry lookup	Registered owner, vessel or aircraft record, contact details (redacted before any external model call)	Prefetched by code from the decoded beacon identifier
Alert and activation history	Prior alerts from the same beacon and how each was resolved	Prefetched by code
Other detections	Other satellite or MCC detections of the same beacon in the time window	On demand, Tiers 2 and 3
Vessel or aircraft track	Position reports near the detected location (for example AIS or ADS-B)	On demand, Tiers 2 and 3
Weather and sea state	Conditions at the detected position and time	On demand, Tiers 2 and 3
Drift and search-area estimate	A read-only computation from position, time and conditions	On demand, when severity indicators are present

The list illustrates the pattern. Which sources exist depends on each hub's data agreements, and the table does not describe what any MCC operates.

V. THE ECONOMICS

A. Price Points Used

Prices are per 1 million tokens, taken from each provider's published pricing [2]–[4]. They are list prices of earlier model generations, so treat the dollar figures as a worked example of the method. Verify them against current provider documentation before budgeting, as listed prices and model generations change often.

The model uses GPT-4o Mini for Tier 1 and Claude 3.5 Sonnet for Tier 2. Sonnet's input costs 20 times Mini's and its output 25 times, and that gap is the economic basis of routing.

B. Three Architectures Compared

Architecture A is a deterministic baseline with no failure penalty. B and C pay for every escalation, including the wasted first attempt. The comparison is therefore tilted against the proposed design on failure accounting, and the savings are conservative in that respect (Table V). These three architectures price the tool-free workflow, with the registry record and history prefetched into the goal; Section VI adds tool calls.

Fig. 2 shows where each architecture's money goes. Input tokens are 77.2% of the baseline bill, which is why routing and caching, both of which attack input cost, are so effective. In architecture C the wasted first attempts cost only about $1.75 per 1,000 assessments.

C. Where the Savings Come From

- **Routing alone: 34.6% reduction** ($0.1182 to $0.0773). Half the steps run on a model about 20 times cheaper on input.
- **Caching on top of routing: a further 51.4% reduction** ($0.0773 to $0.0376). The 6,000-token static prompt is re-read on all four steps, and in architecture C a cached read serves 78.8% of all input tokens (26,400 of 33,521 per run).
- **Combined: 68.2% reduction**, or **$235,465 a year** for one workflow. Against a baseline that is also charged the 5% premium-model failure rate, the combined saving is 69.9%.
- **The escalation tax.** A pipeline whose cheap steps never failed would cost $0.0312 per run (a 73.6% reduction). Real escalations raise that by 20.4%, with about 0.40 escalation or retry events per run. The business gets valid outputs in exchange.

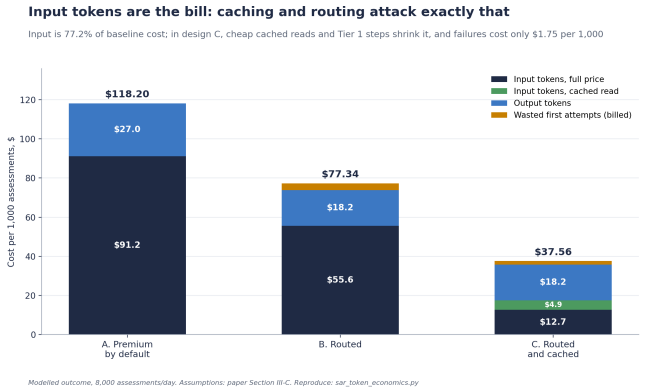


Fig. 2. Cost per 1,000 assessments by architecture, split into full-price input, cached input, output and billed first attempts that failed. Modelled outcome at 8,000 assessments a day (Section III-C).

Fig. 3 shows the annual bridge from the premium baseline to the routed and cached design. Each lever contributes roughly half of the $235,465 saving. The saving also scales linearly with volume: doubling the alert-assessment rate doubles the dollar saving without changing the architecture.

D. Robustness: Do the Savings Survive Worse Assumptions?

The two assumptions most likely to be wrong in a real deployment are the cache hit rate and the Tier 1 failure rate. Both were varied with all else held fixed (Tables VI and VII).

Fig. 4 plots both sweeps. The saving falls steadily as either assumption worsens, but it does not collapse.

Two points of interpretation. First, the figure 51.4% appears twice for unrelated reasons: caching removes 51.4% of the routed cost (Section V-C), and a 50% cache hit rate leaves a 51.4% saving against the premium baseline. Second, because escalation re-runs the same payload on the premium model, a higher failure rate mainly raises cost rather than changing what the controller receives; shadow testing should confirm this. Routing alone (without caching) only becomes more expensive

TABLE IV
LIST PRICES USED IN THE MODEL (USD PER 1M TOKENS)

Model	Input	Output	Cached input	Notes
OpenAI GPT-4o	$2.50	$10.00	$1.25	50% discount on cached inputs
Claude 3.5 Sonnet	$3.00	$15.00	$0.30	90% savings via prompt caching
Gemini 1.5 Pro	$1.25	$5.00	n/a	Context caching lowers base rates; higher rates apply to very long prompts
GPT-4o Mini	$0.15	$0.60	$0.075	Cost-effective for agent micro-steps
Gemini 1.5 Flash	$0.35	$1.50	n/a	Built for speed and ultra-low cost

TABLE V
COST OF THE THREE ARCHITECTURES (EXACT EXPECTATION OVER ALL FAILURE OUTCOMES)

Architecture	Description	Per run	Per 1,000 runs	Per day	Per year
A. Premium by default	All four steps on Claude 3.5 Sonnet, no caching	$0.1182	$118.20	$946	$345,144
B. Routed	Tier 1 / Tier 2 split with escalation, no caching	$0.0773	$77.34	$619	$225,836
C. Routed and cached	B plus caching of the static prefix	$0.0376	$37.56	$300	$109,679

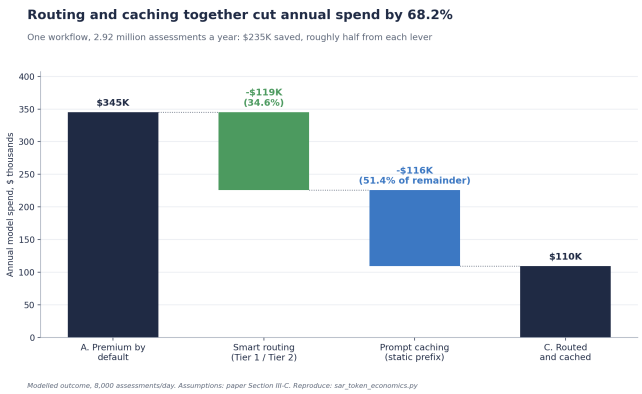


Fig. 3. Annual model spend for one workflow (2.92 million assessments a year): $345K falls to $110K, with routing saving $119K and caching a further $116K. Modelled outcome (Section III-C).

TABLE VI
SENSITIVITY TO THE SHARE OF THE STATIC PREFIX SERVED FROM CACHE

Share from cache	Cost per run	Saving vs A
100%	$0.0376	68.2%
90%	$0.0415	64.9%
75%	$0.0475	59.8%
50%	$0.0575	51.4%
0% (routing alone)	$0.0773	34.6%

TABLE VII
SENSITIVITY TO THE TIER 1 FIRST-ATTEMPT FAILURE RATE

Tier 1 failure rate	Cost per run	Saving vs A
5%	$0.0345	70.8%
15% (base case)	$0.0376	68.2%
30%	$0.0421	64.4%
45%	$0.0467	60.5%
60%	$0.0513	56.6%

than premium-by-default when Tier 1 fails about **79.9%** of the time: at a 100% failure rate the routed pipeline costs $0.1309 per run against $0.1182.

E. Tail Risk

Averaging hides danger. The exact 99th-percentile run in architecture C costs $0.0683, which is 1.82 times the mean, and the worst possible combination of failures costs $0.1010. The larger risk is a pathological run: if an agent hits the 30-times volatility described in Section II-C, a baseline run costs about $3.55 and the same event in architecture C costs about $1.13. A per-run ceiling (for example five times the mean, about $0.19) converts that unbounded exposure into a bounded one. Under invariant 4, hitting the ceiling hands the case to a human; it never discards it.

F. Counting Convention

A simpler convention does not charge for a failed first attempt. Following that convention, architecture C would cost $0.0358 per run, 4.7% less than the $0.0376 reported here. This paper uses the higher figure.

VI. THE ECONOMICS OF THE TOOL LAYER

A. Why Tools Change the Arithmetic

Each tool call is paid for twice. The model emits a request and the gateway returns a result, and then the next model turn must re-read the whole context, including that result, before it can act. A step with k tool rounds therefore makes $k + 1$ model calls, and the result of round j is re-read on each of the $k - j + 1$ calls that follow it. Input grows with the square of the number of rounds. This is the state-carrying mechanism of Section II-C, now driven by the agent's own choices instead of a fixed pipeline.

B. Added Assumptions

All assumptions of Section III-C still apply. The tool layer adds the following, which are as illustrative as the rest.

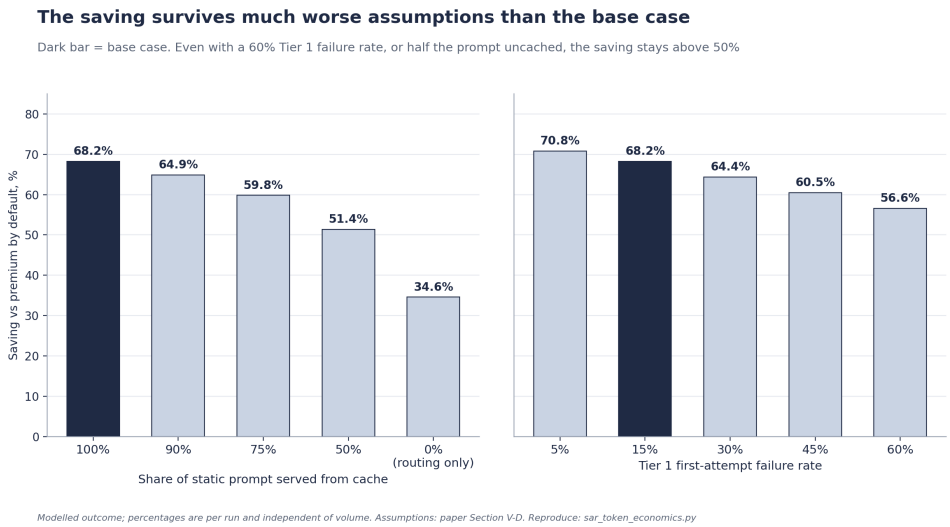


Fig. 4. Saving versus premium by default as the cache share (left) and the Tier 1 failure rate (right) vary. Dark bars mark the base case. Modelled outcome (Section V-D).

- Tool definitions of 1,200 tokens join the Step 2 prefix. They are static, so caching covers them.
- Step 2 makes three sequential tool rounds in the base case, one call per round, on the Tier 2 model. Each call request costs 100 output tokens.
- The gateway returns 500 tokens per result. Without the gateway the same result is 1,500 tokens.
- With the gateway, Steps 3 and 4 receive a 100-token evidence digest per call. A naive agent carries the full transcript forward.
- If the final assessment fails validation, the wasted call is billed and the call is re-run with 40% more output, as in Section III-C.
- The per-run ceiling is eight tool rounds.
- Not modelled: fees charged by data providers, tool latency and tool failures (Appendix B).

C. Cost of the Agentic Design

Table VIII compares the tool-using designs with the two workflows of Section V-B. The naive premium agent (A1) runs every step on the premium model, receives raw tool results, carries the whole transcript forward and uses no caching. Design C1 is the proposal: routing, caching and the gateway.

The proposed design costs 72.2% less than the naive premium agent: $210,463 against $757,740 a year, a saving of $547,277. Applied in order, the gateway removes 17.3%, routing a further 19.1% of the remainder, and caching a further 58.5% of what is left. The order matters and the shares are not additive. Caching carries most of the saving here, because tool results enlarge the context and the 7,200-token prefix is re-read on every turn: a cached read serves 76.5% of design C1's input tokens (49,260 of 64,385 per run).

The tool tax. Against the tool-free workflow C, tools add $34.51 per 1,000 assessments, a rise of 91.9%, or about $100,800 a year. That is the price of letting the model decide what evidence to fetch, and Section VIII says how to judge whether it is worth paying. Design C1 is still 39.0% cheaper than the tool-free premium workflow A, so the cheapest tool-using design costs less than the simplest design before routing and caching. With zero tool rounds the model collapses to workflow C within 0.8% ($0.0379 against $0.0376), which is the cost of carrying the tool definitions.

D. Cost Grows Faster than the Number of Rounds

Fig. 5 and Table IX vary the number of tool rounds in Step 2. In design C1 each extra round costs about $0.0018 more than the one before it: the first round adds $0.0096 and the tenth $0.0258. Total cost is therefore quadratic in rounds. Design C1 stays below the tool-free premium workflow A ($0.1182) up to five rounds and crosses it at six ($0.1224).

E. Levers Inside the Tool Layer

- **Trim results.** The gateway's trimmed results and digests cut design C1's cost by 32.9% against raw results ($0.1074 to $0.0721). Result size acts directly on cost: each extra 500 tokens per result adds about $0.0092 per assessment in the base case, and results of 3,000 tokens would raise C1 to about $0.1182, the cost of the tool-free premium workflow.
- **Call in parallel.** Independent lookups (other detections, vessel track, weather) can go in one round. Three calls in one round cost $0.0550 per assessment, 23.6% less than three sequential rounds, and also shorten the time to brief.
- **Prefetch what is always needed.** Evidence every assessment requires is fetched by code before the model runs, so it costs no model turns.
- **Protect the cache.** With tools, a weaker cache hurts far more than in the workflow. At 75%, 50% and 0% of the static prefix served from cache, design C1 costs $0.0975,

TABLE VIII
COST OF THE TOOL-USING DESIGNS AT THREE TOOL ROUNDS (EXACT EXPECTATION)

Design	Description	Per run	Per 1,000 runs	Per year
Workflow A	Premium by default, no tools (Table V)	$0.1182	$118.20	$345,144
Workflow C	Routed and cached, no tools (Table V)	$0.0376	$37.56	$109,679
A1. Naive premium agent	All steps premium, raw results, full transcript carried, no caching	$0.2595	$259.50	$757,740
A1 with gateway only	A1 plus trimmed results and evidence digests	$0.2145	$214.50	$626,340
B1. Routed, gateway	Tier 1 / Tier 2 split plus gateway, no caching	$0.1736	$173.58	$506,848
C1 without gateway	Routed and cached, raw results, full transcript carried	$0.1074	$107.40	$313,612
C1. Routed, cached, gateway	**The proposed design**	**$0.0721**	**$72.08**	**$210,463**

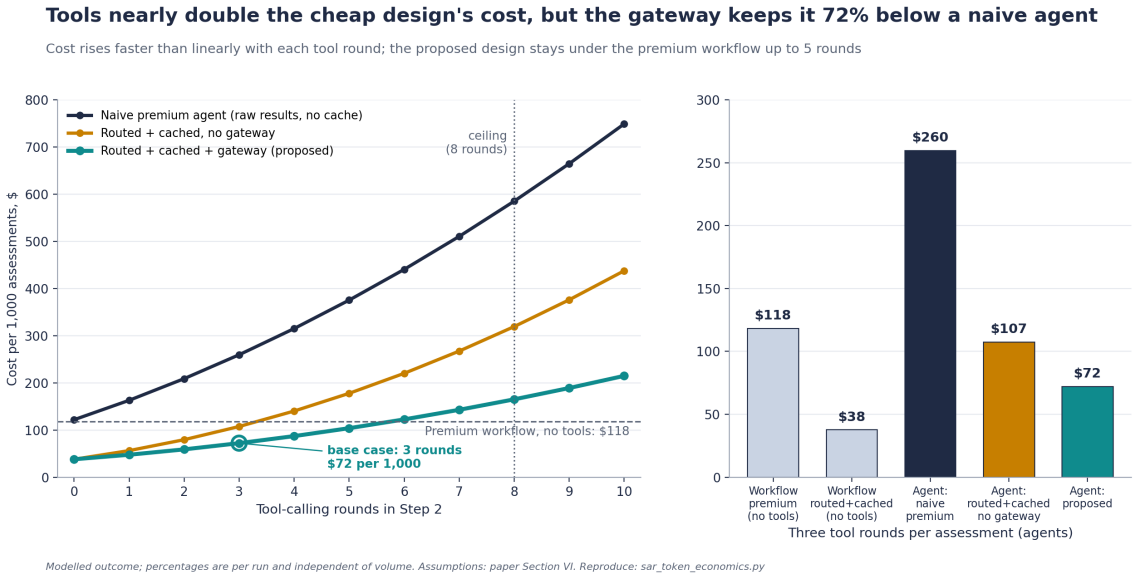


Fig. 5. Left: cost per 1,000 assessments as the number of tool rounds in Step 2 grows, for a naive premium agent, a routed and cached agent without the gateway, and the proposed design. Right: the five designs at three rounds. Modelled outcome (Section VI).

TABLE IX
COST PER RUN (USD) BY NUMBER OF TOOL ROUNDS

Rounds	Naive premium	C1, no gateway	C1 (proposed)
0	0.1218	0.0379	0.0379
1	0.1629	0.0563	0.0475
3	0.2595	0.1074	0.0721
5	0.3753	0.1777	0.1038
8	0.5850	0.3192	0.1650
10	0.7488	0.4375	0.2147

$0.1228 and $0.1736 per assessment, which is 52.7% below A1 at 50% and 33.1% below it with no caching at all. In the workflow the same 50% cache share cost $0.0575.

F. Tail Risk with Tools

The ceiling matters more once the model chooses its own actions. The exact 99th-percentile run in design C1 costs $0.1089, 1.51 times the mean, and the worst combination of failures costs $0.1435. A run that uses all eight permitted rounds costs $0.1650, or 2.29 times the mean. Without a ceiling, an agent that loops for 30 rounds (an illustrative runaway, not a measured one) costs $1.11 with the gateway and $2.68 as a naive agent, which is 37.1 times the C1 mean. The ceiling turns that exposure into a bounded one. Under invariant 4, reaching it hands the case to a human with the evidence gathered so far.

G. What the Numbers Do Not Show

The model prices tool use; it does not value it. Whether the extra evidence improves a controller's decision is an empirical question for the pilot (Section VIII). Sequential rounds also add latency to the time to brief, roughly one model round trip plus the tool's own response time per round, which parallel calls and prefetching reduce. Provider fees and tool failures are not modelled. Treat three rounds as a planning assumption, not a measurement. Section IX brings tools in only after routing has been measured, so that the number of rounds actually used can replace it.

TABLE X
CUMULATIVE INPUT TOKENS WITH AND WITHOUT COMPACTION

Steps	Plain	With compaction	Reduction
4	30,600	30,600	0%
10	91,500	91,500	0%
20	233,000	193,000	17%
40	666,000	396,000	41%

TABLE XI
COST PER RUN (USD) BY RUN LENGTH AND OPTIMISATION

Steps	Plain	Caching	Compaction	Both
4	0.122	0.057	0.122	0.057
10	0.350	0.188	0.350	0.188
20	0.849	0.525	0.729	0.405
40	2.298	1.650	1.488	0.840

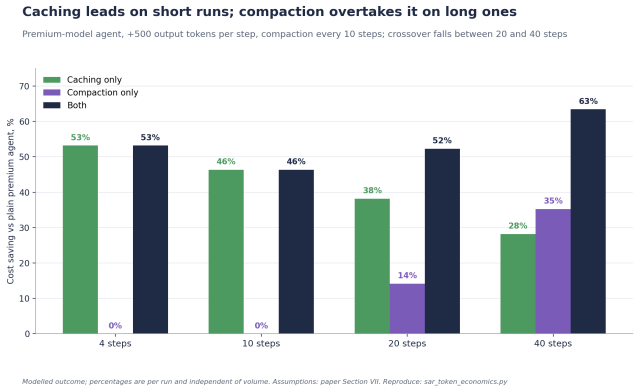


Fig. 6. Cost saving versus a plain premium-model agent at 4, 10, 20 and 40 steps, for caching only, compaction only, and both. Modelled outcome (Section VII).

VII. LONG-RUNNING INVESTIGATIONS: WHY COMPACTION MATTERS

The four-step triage run is short. Some SAR support tasks are not: an agent that follows an evolving case, re-assessing as new detections arrive and drafting situation updates, may run for dozens of steps. The tables below model a premium-model agent with a 6,900-token base context (6,000 static plus 900 goal), 500 tokens of output carried forward at each step, and compaction every 10 steps that replaces history with a 1,000-token summary.

Ten times as many steps (4 to 40) produce 21.8 times as many input tokens. Because each step re-reads everything before it, input grows roughly with the square of the step count, so cost grows faster than linearly with agent autonomy.

Caching saves 53%, 46%, 38% and 28% across these four run lengths. Compaction saves 0%, 0%, 14% and 35%. Together they save 53%, 46%, 52% and 63% (Fig. 6).

Architectural insight: the static prefix is a large share of input in short runs, so caching leads there. As history accumulates, the cached portion shrinks relative to the total and compaction overtakes it; in this scenario the crossover falls between 20 and 40 steps. The two levers are complementary. Short, high-volume triage is caching-led, and long case-following agents need both.

VIII. GOVERNING FOR VALUE, NOT VOLUME

Tokens measure cost, not outcomes. The cost side here is small: $37.56 per 1,000 assessments for the tool-free workflow, and $72.08 with tools enabled. The value case therefore rests on controller time saved and uncertainty reduced, and those benefits must be measured in a pilot rather than assumed. I recommend replacing token leaderboards with the measures in Table XII.

Sizing the recall gate. A target of no misses is only meaningful if the validation set is large enough to test it. If a set containing n genuine alerts shows zero misses, the 95% upper confidence bound on the true miss rate is roughly $3/n$ (the rule of three). Genuine alerts are about one in 30, so a set of 300 genuine alerts, which bounds the miss rate near 1%, needs roughly 9,000 historical alerts. Plan the validation set, and the length of the shadow phase, around that arithmetic.

Before moving any step to a smaller model, test whether it reaches the same operational outcome at lower cost. A saving that degrades a distress judgement is not a saving. And never reward consumption: reward resolved, well-assessed alerts at lower unit cost.

IX. IMPLEMENTATION ROADMAP

Table XIII gives a phased plan. Caching and instrumentation come first because they carry the least quality risk. Routing is introduced only once measurement is in place, and tools come after routing, because every tool round multiplies the cost of any weakness upstream (Section VI).

X. RISKS AND MITIGATIONS

Table XIV lists the principal risks of deploying the agent and the mitigation for each. The first row is the one that matters most: a genuine distress that the agent ranks too low. The safety invariants in Section IV-G exist chiefly to contain it. The remaining rows fall into four groups: resilience (outage, tool failure, runaway loops), data protection and security (personal data, hostile text in alerts or tool output, fabricated context), human factors (automation bias), and cost-control assumptions (escalation storms, tool-loop cost, cache invalidation and shortfall, compaction, price churn). Mitigations that depend on measurement, such as the cache hit ratio and the escalation rate, rely on the telemetry in Section IV-E and are put in place during Phases 1 to 3 of the roadmap (Section IX).

XI. CONCLUSION

Cospas-Sarsat alert handling is a place where AI assistance can matter, because the work is high-volume, evidence-heavy and dominated by alerts that turn out not to be emergencies. It is also a place where cost optimisation must be subordinate to safety, and where an unmanaged agent can quietly become a runaway expense.

TABLE XII
RECOMMENDED GOVERNANCE METRICS

Metric	Definition	Why it matters
Genuine-alert recall	Share of genuine alerts in the validation set that the agent ranks above the controller-review threshold	A release gate: the target is no misses
Cost per assessed alert	Total model spend divided by assessments delivered	The headline unit economics ($0.0376 in the base case)
Controller agreement and override rate	How often controllers accept the agent's likelihood ranking and recommended checks	Shows whether the agent is actually useful, and whether trust is calibrated
Time to brief (p95)	Latency from alert receipt to case brief	Slow briefs are worthless in an emergency; measure it per tier
Escalation rate	Share of Tier 1 steps re-routed upward	Early warning that a Tier 1 step is mis-scoped
Cache hit ratio	Cached input tokens as a share of all input tokens	Confirms prompt discipline (78.8% in the base case, which assumes a 100% hit rate on the static prefix)
Tail-run ratio	99th-percentile run cost divided by the mean	Quantifies volatility exposure (1.82 in the base case)
Tool rounds per assessment	Mean and 99th-percentile number of tool-calling rounds in Step 2	The main driver of agentic cost: the third round adds $0.0132 in the base case, and every later round costs more
Grounding rate	Share of statements in the brief that cite a tool result or a supplied record	Enforces invariant 8; a falling rate means the agent is filling gaps from memory
Tool failure rate	Share of tool calls that time out, error or return nothing	Missing evidence degrades the brief and wastes rounds

TABLE XIII
PHASED IMPLEMENTATION ROADMAP

Phase	Timeline	Activities	Exit criterion
0. Safety case	Weeks 1 to 3	Agree the eight invariants with MCC and RCC operators; data-protection and legal review; define the recall target and validation set	Signed safety case and approved data-handling plan
1. Instrument and shadow	Weeks 3 to 6	Run the agent in shadow mode with no operator exposure; log per-call tokens and cost; compare its rankings with actual outcomes	Full cost visibility and a measured baseline cost per assessed alert
2. Cache	Weeks 6 to 8	Restructure prompts as static prefix plus dynamic suffix; enable provider caching; track hit ratio	Cache hit ratio at target with no change in output quality
3. Route	Weeks 8 to 12	Classify steps by complexity; move Tier 1 steps to lite models; add schema validation and escalation; shadow-test against the premium baseline	Genuine-alert recall unchanged; accuracy at tier within tolerance
4. Add read-only tools	Weeks 12 to 16	Build the gateway; start with prefetched lookups only; enable on-demand tools one at a time in shadow mode; measure rounds per assessment, grounding rate and tool failures; set the tool-call ceiling from measured use	Grounding rate at target; cost per assessed alert measured at the real number of rounds; recall unchanged
5. Operator pilot	Weeks 16 to 22	Show briefs to controllers at one RCC; measure time to brief, agreement and override rates	Controllers confirm usefulness; no safety regressions
6. Compact, cap and scale	Ongoing	Add compaction for long investigations; set budget ceilings with fail-open behaviour; publish dashboards; extend to further RCCs	Control plane adopted as a shared platform

The Token-Aware Control Plane answers both concerns. Route each step to the cheapest tier that can safely do it, cache what never changes, compact what keeps growing, escalate when a cheap model fails, and measure everything against outcomes. In the modelled deployment, at 2024-era list prices, these disciplines cut annual model spend by 68.2%, about $235,000 for one workflow, while the safety invariants keep humans in command and the alert path untouched.

Making the agent genuinely agentic changes the economics but not the argument. Letting Step 2 call read-only tools through a gateway nearly doubles the cost of the cheap design, to $0.0721 per assessment, because every tool round re-reads the whole context. The same disciplines, with a gateway that trims results and a ceiling on rounds, keep that 72.2% below a naive tool-using agent. Whether the tools earn their cost is for the pilot to show, in controller time and certainty rather than in tokens.

The question for any AI initiative in this field is not "how many tokens did we use?" but **"what did each assessed alert cost, what did it save a controller, and did it ever make a genuine distress harder to see?"** An architecture that can answer all three is one that rescue services can trust at scale.

APPENDIX A
REPRODUCING EVERY FIGURE

Two files accompany this paper: `sar_token_economics.py` (Python 3.9 or later, standard library only) and `expected_output.txt` (the output captured when the paper was written). Both

TABLE XIV
RISKS AND MITIGATIONS

Risk	Mitigation
A genuine distress is ranked too low	Advisory-only agent; severity floor; recall gate on every release; humans decide; alert forwarding is unchanged
Agent or provider outage	Off the critical path; degraded mode is normal operations; multi-provider failover with at least two qualified models per tier
Exposure of personal data (beacon registration records include owner contact details)	Minimise and redact before any external call, including tool arguments and results; approved regions or private endpoints; retention limits; legal review per jurisdiction
Malformed or hostile text in alerts, registry fields, retrieved documents or tool output	Treat all retrieved text and tool output as data, never instructions; the gateway delimits results and logs suspected instructions; strict schema validation; read-only tool permissions
Fabricated context presented as fact	Ground every statement in a retrieved record and show the source to the controller; verification step before display
Automation bias (controllers over-trust the brief)	Show uncertainty and sources; train controllers; run periodic blind audits
Escalation storms (cheap model fails often, premium model is paid for every step)	Alert on escalation rate; re-scope or re-prompt the Tier 1 step; cap retries
Cache invalidation from prompt edits	Version and freeze the static prefix; change it only through controlled releases
Cache savings fall short of the 100% base case (lower hit rate, cache expiry between calls, or a cache-write fee)	Measure the hit ratio in Phase 2 and budget with the measured value; Section V-D shows the saving is still 51.4% at a 50% hit rate
Over-aggressive compaction drops critical facts	Protected fields outside the compactable region; recall tests on compacted runs
Price and model churn	Keep the model layer behind the router; re-run the cost script every quarter with current prices
Tool failure, timeout or stale data	Gateway timeouts inside the ceiling; the brief lists missing evidence and shows data timestamps; the agent never fills a gap from model memory; degraded mode applies
Tool use inflates cost and latency	Per-run tool-call ceiling; parallel calls for independent lookups; trimmed results and evidence digests; alert on rounds per assessment (Section VI)
Runaway loops	Per-run token ceiling, step caps, tool-call ceiling and circuit breakers that fail open to a human

files are published, with a README and an MIT licence, in the repository cited as [11] (release v1.1): https://github.com/smruti1979/sar_token_economics. The figure scripts, `draw_charts.py` and `draw_architecture.py`, import the same cost model. To reproduce the figures, run the script as shown below.

```
python3 sar_token_economics.py --selftest
python3 sar_token_economics.py
python3 sar_token_economics.py --verify
python3 sar_token_economics.py --json
python3 sar_token_economics.py --runs-per-day 20000 --
    static-tokens 8000 --cache-hit 0.75
python3 sar_token_economics.py --tool-rounds 5 --result-
    tokens 1000
```

The commands run, in order, the sanity checks on the arithmetic, the full report, an assertion of the paper's headline numbers, machine-readable output, a custom workflow scenario, and a custom tool-layer scenario. Table XV maps each part of the script output to the paper.

TABLE XV
SCRIPT OUTPUT AND PAPER SECTIONS

Script output	Paper section
[1] Three architectures	V-B, V-C, I
[2] Monte Carlo cross-check (agrees with the exact result to within 0.03%)	III-C
[3] Tail risk	V-E
[4] and [5] Sensitivity, and break-even	V-D
[6] Long-running agent	VII
[7] Tool layer, with its own Monte Carlo check (within 0.02%)	VI

Method.

1) At each step, input equals everything in the context window: the static prompt, the goal and all previous step outputs.
2) Step cost is

$$C = \frac{n_u\, p_i + n_c\, p_c + n_o\, p_o}{10^6}, \tag{1}$$

where n_u, n_c and n_o are uncached input, cached input and output tokens, and p_i, p_c and p_o are the corresponding prices per million tokens.
3) With caching, the static prefix is billed at the cache-read price and the remainder at the standard input price.
4) In the tool layer, Step 2 becomes k tool-calling model calls followed by one assessment call. Each round adds the call request and its result to the context re-read by every later call in the step. With the gateway, only a short digest per call is carried into Steps 3 and 4.
5) If a step fails validation, the wasted attempt is billed, then the step re-runs (on the premium model if it began on Tier 1) with 40% more output.

6) The expected cost is the probability-weighted sum over all 16 failure outcomes of the four steps, so no sampling noise enters the headline figures.

The `--selftest` command checks a hand-calculated premium call ($0.0252 uncached, $0.0090 with caching), that outcome probabilities sum to 1, and that the Monte Carlo result stays within 1% of the exact one. For the tool layer it also checks that zero tool rounds reproduce the workflow exactly, a hand-calculated one-round premium agent ($0.1509), that cost rises with every extra round, and that the gateway never costs more than raw results. The `--verify` command asserts 21 headline numbers from this paper. Because the paper reports the default scenario, `--verify` will fail by design if you change an assumption; use the plain report to explore other scenarios.

APPENDIX B
DATA NOTES

- **Prices and models.** The price table is in Section V-A. The model uses GPT-4o Mini and Claude 3.5 Sonnet. Provider pricing changes frequently and newer model generations may be available, so update the `PRICES` table in the script and re-run it before publication or budgeting.
- **Not modelled:** the extra fee some providers charge for writing to a cache (as opposed to reading from it); cache expiry between calls; Tier 3 reasoning-model costs; fees charged by tool and data providers; tool failures; latency; integration, hosting and engineering effort; and the cost of evaluation and monitoring.
- **Tool-layer inputs** (three rounds, 500-token results, 1,200-token tool definitions, an eight-round ceiling) are assumptions, and the tool set in Table III is illustrative.
- **Scenario inputs are assumptions.** Volume, prompt sizes, output sizes and failure rates are illustrative and should be replaced with measured pilot values. Cost reductions are modelled outcomes, not guarantees.
- **Cospas-Sarsat facts** were checked against the Secretariat's status papers and presentations to ICAO meetings [5]–[10]. The most recent SAR false alert rate found is for 2020 (95.55%); check for newer figures before publication.
- **Provider prices** are from the providers' pricing documentation [2]–[4]. Those pages show current prices, while the prices in this paper are 2024-era, so record the retrieval date and, where a model has been retired, cite an archived copy of the page.

REFERENCES

[1] L. Bai *et al.*, "How do AI agents spend your money? Analyzing and predicting token consumption in agentic coding tasks," Stanford Digital Economy Lab, 2026. [Online]. Available: https://digitaleconomy.stanford.edu/publication/how-do-ai-agents-spend-your-money-analyzing-and-predicting-token-consumption-in-agentic-coding-tasks/

[2] OpenAI, "API pricing." [Online]. Available: https://openai.com/api/pricing/

[3] Anthropic, "Claude API pricing." [Online]. Available: https://www.anthropic.com/pricing

[4] Google, "Gemini API pricing." [Online]. Available: https://ai.google.dev/gemini-api/docs/pricing

[5] Cospas-Sarsat Secretariat, "Status of the Cospas-Sarsat Programme," ICAO APSAR/WG/11, working paper, 2026. [Online]. Available: https://www.icao.int/sites/default/files/APAC/Meetings/2026/2026%20APSAR WG11/03-Working%20Papers/AI03%20WP04%20Status%20of%20the%20Cospas-Sarsat%20Programme.pdf

[6] Cospas-Sarsat Secretariat, "Status of the Cospas-Sarsat Programme," presentation, ICAO EUR/NAT SAR Workshop, Paris, Nov. 2025. [Online]. Available: https://www.icao.int/sites/default/files/EURNAT/MeetingDocs/SAR%20Meetings%20and%20Workshops/2025%20Joint%20ICAO%20EURNAT%20and%20ACAO%20SAR%20Workshop%20/PPT05-Status-of-the-COSPAS-SARSAT-Programme.pdf

[7] Cospas-Sarsat Secretariat, "Status of the Cospas-Sarsat Programme," presentation, ICAO EUR/APAC SAR Workshop, Baku, Oct. 2024. [Online]. Available: https://www.icao.int/sites/default/files/EURNAT/MeetingDocs/SAR%20Meetings%20and%20Workshops/2024%20EUR%20APAC%20Search%20and%20Rescue%20Workshop/PPT07.pdf

[8] Cospas-Sarsat Secretariat, "Status of the Cospas-Sarsat Programme," ICAO APSAR/WG/10, working paper WP14, 2025. [Online]. Available: https://www.icao.int/sites/default/files/APAC/Meetings/2025/2025%20APSARWG10/3-Working%20Papers/WP14-Status-of-the-Cospas-Sarsat-Programme.pdf

[9] Cospas-Sarsat Secretariat, "Addendum: False alert rate," ICAO APSAR/WG/6, working paper WP05, 2021. [Online]. Available: https://www.icao.int/sites/default/files/APAC/Meetings/2021/2021%20APSAR%20WG6/3-Working%20Papers/WP05-Addendum-False-Alert-Rate.pdf

[10] Cospas-Sarsat Secretariat, "Report on Cospas-Sarsat system status," ICAO NACC SARTF/2, SARTF02-NE02, 2021. [Online]. Available: https://www.icao.int/NACC/Documents/Meetings/2021/SARTF2/SARTF02-NE02.pdf

[11] S. Rout, "sar_token_economics: Reproducible cost model for a Cospas-Sarsat alert intelligence agent," GitHub repository, release v1.1, 2026. [Online]. Available: https://github.com/smruti1979/sar_token_economics